

GUÍA TÉCNICA — Cómo ejecutar un proyecto Angular localmente (con backend Spring Boot y base de datos Docker)

1. Requisitos previos

Antes de iniciar, asegúrate de tener instaladas las siguientes herramientas:

Herramienta	Descripción	Verificar instalación
Node.js (v18 o superior)	Permite ejecutar Angular y npm	<code>node -v</code>
npm (Node Package Manager)	Gestiona dependencias del proyecto	<code>npm -v</code>
Angular CLI (v16 o superior)	Herramienta de línea de comandos de Angular	<code>ng version</code>
Visual Studio Code	Editor recomendado	—
Docker	Para levantar la base de datos del backend	<code>docker -v</code>

Si no tienes Angular CLI instalado, ejecútalo:

```
npm install -g @angular/cli
```

2. Abrir el proyecto en Visual Studio Code

Ubícate en la carpeta donde se encuentra tu proyecto Angular (por ejemplo, `frontend/`).

Abre una terminal en esa carpeta y ejecuta:

```
code .
```

Esto abrirá Visual Studio Code directamente en tu proyecto Angular.

3. Iniciar la base de datos con Docker (del backend)

Antes de levantar el frontend, asegúrate de que el backend y la base de datos estén funcionando.

Desde la carpeta raíz de tu backend (donde está el `docker-compose.yml`), ejecuta:

```
docker-compose up -d
```

Verifica que los contenedores estén activos:

```
docker ps
```

Si todo está correcto, el backend debería estar escuchando en `http://localhost:8080`.

4. Verificar conexión entre Angular y el backend

En tu proyecto Angular, revisa el archivo de configuración donde se define la URL del backend. Normalmente está en:

`src/environments/environment.ts`

```
export const environment = {  
  production: false,  
  apiUrl: 'http://localhost:8080/api' // URL del backend Spring Boot  
};
```

Y en producción:

`src/environments/environment.prod.ts`

```
export const environment = {  
  production: true,  
  apiUrl: 'https://mi-backend-produccion.com/api'  
};
```

Asegúrate de que la URL de `apiUrl` coincida con la que expone tu backend Docker (por ejemplo, `http://localhost:8080`).

5. Instalar dependencias del proyecto Angular

En la raíz del proyecto Angular, ejecuta:

```
npm install
```

Esto descargará e instalará todas las dependencias necesarias que están listadas en el archivo `package.json`.

Este paso puede tardar algunos minutos la primera vez.

6. Ejecutar el proyecto Angular localmente

Una vez que las dependencias estén instaladas, ejecuta el siguiente comando:

```
ng serve
```

Por defecto, Angular se ejecutará en el puerto 4200.

Abre tu navegador y accede a:

```
http://localhost:4200
```

Deberías ver tu aplicación Angular funcionando y conectándose al backend.

7. Personalizar el puerto (opcional)

Si el puerto 4200 está ocupado, puedes levantar el servidor en otro puerto con:

```
ng serve --port 4300
```

Y acceder en tu navegador a:

```
http://localhost:4300
```

8. Comprobación de comunicación con el backend

Si tu frontend realiza llamadas HTTP (por ejemplo, con `HttpClient`), verifica que los endpoints respondan correctamente.

Ejemplo en el navegador o consola de red:

```
GET http://localhost:8080/api/clientes
```

Si obtienes respuesta, significa que tu frontend y backend están comunicándose correctamente.

9. Detener los servicios

Cuando termines de trabajar:

Detén el servidor Angular con:

```
Ctrl + C
```

Apaga la base de datos y el backend con:

```
docker-compose down
```

10 Recomendaciones finales

Usa los archivos de entorno (`environment.ts` y `environment.prod.ts`) para cambiar fácilmente entre entornos (local, QA, producción).

Si tu backend usa CORS, asegúrate de permitir el origen del frontend local.
En Spring Boot, puedes hacerlo agregando una configuración global:

```
@CrossOrigin(origins = "http://localhost:4200")
```

Para un entorno completo:

- Levanta backend y base de datos con Docker.
- Ejecuta frontend con `ng serve`.
- Usa Postman o el navegador para probar los endpoints.